AUTOMATED SECURITY ASSESSMENT & REPORTING PIPELINE

รายงานการแก้ไขระบบ ขั้นที่ 5

REST API · API key · คิวงานด้วย Redis · worker · กันการโจมตีผ่าน API · ระบบ login

ต่อจาก "รายงานการแก้ไขระบบ ขั้นที่ 4"

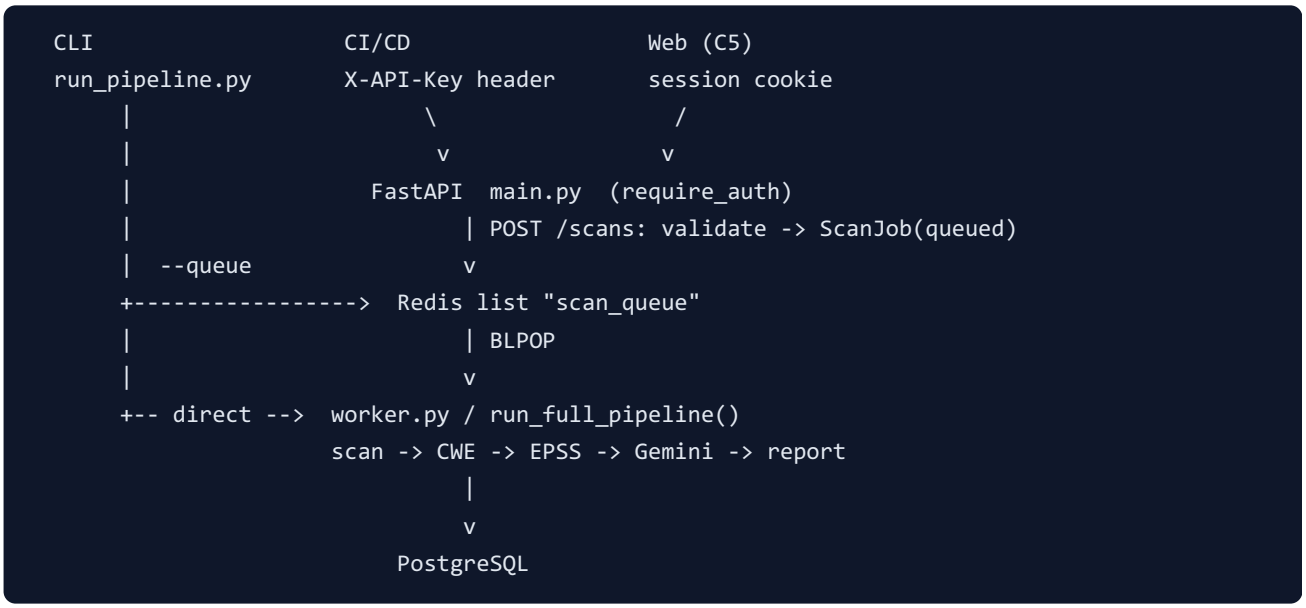
> โปรเจกต์: `C:\Allprojects\Projects`
> GitHub: `github.com/jjjecop/security-pipe-ine` (private)
> ช่วงเวลา: 11–13 ตุลาคม 2026 · test ทั้งหมด 83 ตัว

1. สรุปสั้นๆ

ขั้นที่ 1–4 ทำให้ระบบ**สแกนและรายงานได้ถูกต้องจากคำสั่งเดียว** ส่วนขั้นที่ 5 ทำให้ระบบ**ถูกเรียกใช้จากภายนอกได้อย่างปลอดภัย** ทั้งจาก CI/CD (ใช้ API key) และจากหน้าเว็บในอนาคต (ใช้ user/password) โดยงานสแกนที่ใช้เวลานานถูกส่งไปทำเบื้องหลังผ่านคิว

เรื่อง	ก่อน	หลัง
ดูผลสแกน	เปิด PDF หรือ query ฐานข้อมูลเอง	REST API: โปรเจกต์, ช่องโหว่ (กรอง/แบ่งหน้า), สถานะ ScanJob, ดาวน์โหลด PDF
สั่งสแกน	พิมพ์คำสั่งใน terminal แล้วรอจนเสร็จ	`POST /scans` ตอบทันที (202) แล้ว worker สแกนเบื้องหลัง
ใครเข้าถึงได้	เฉพาะคนที่นั่งหน้าเครื่อง	ต้องมี API key หรือ login ด้วย user/password
ค่าอันตรายจากภายนอก	ไม่มีช่องทางให้ส่งเข้ามา	ตรวจ 5 ด่านก่อนถึง scanner: กัน path traversal, SSRF, argument injection
รหัสผ่าน	ไม่มีระบบ user	Argon2id + กฎความยาว + จำกัดการเดารหัส
ใครสั่งสแกนอะไร	`triggered_by = "manual"` ทุกงาน	`cli` / `api` / `web:admin`
โค้ดแกนกลาง	ขั้นตอนทั้งหมดอยู่ใน `main()` ของ CLI	`run_full_pipeline()` ที่ CLI และ worker ใช้ร่วมกัน
จำนวน test	33	83

ภาพรวมสถาปัตยกรรมตอนนี้



2. C3.1–C3.2: REST API และ API key

1 API อ่านข้อมูล

ทำไมต้องมี API

เปิดช่องให้โปรแกรมอื่นถามข้อมูลผ่าน HTTP ได้ เช่น หน้าเว็บ dashboard, CI ของโปรเจกต์อื่น หรือสคริปต์ที่ถามว่า "ตอนนี้มีช่องโหว่ critical กี่ตัว"

Endpoint

Endpoint	ได้อะไร
`GET /health`	สถานะ API และฐานข้อมูล (`SELECT 1`) ไม่ต้องใช้ key
`GET /projects`	รายชื่อโปรเจกต์และจำนวนช่องโหว่ที่ยัง open
`GET /projects/{name}/findings`	ช่องโหว่เรียงตาม risk score กรองด้วย `status` , `severity` , `tool` แบ่งหน้าด้วย `limit` / `offset`
`GET /scan-jobs/{id}`	สถานะ ScanJob, ผู้สั่ง และผลของ scanner แต่ละตัว

แนวคิดของ FastAPI ที่ได้เรียน

ส่วน	ความหมาย
`@app.get("/projects")`	URL นี้เรียกฟังก์ชันข้างล่าง
`{project_name}`	ส่วนของ URL ที่เป็นตัวแปร
`severity: str | None = None`	query parameter เช่น `?severity=critical`
`Query(50, ge=1, le=500)`	จำกัดค่า ส่ง 1000 มาได้ 422 กันขอข้อมูลทีละล้านแถว
`Depends(get_session)`	FastAPI เปิดและปิดการเชื่อมต่อฐานข้อมูลให้ทุก request
`HTTPException(404)`	ตอบ "ไม่พบ" แบบมาตรฐานแทนการ crash
`/docs`	หน้าคู่มือและทดลองเรียก API ที่ FastAPI สร้างให้อัตโนมัติ

uvicorn

เซิร์ฟเวอร์ที่รับ request แล้วส่งต่อให้ FastAPI รันด้วย `uvicorn main:app --reload` ฟังที่ `127.0.0.1` (เฉพาะเครื่องตัวเอง) เป็นค่าเริ่มต้น

ไฟล์: `main.py` , `requirements.txt` (เพิ่ม `uvicorn`)

2 API key ข้อมูลช่องโหว่คือสิ่งที่ผู้โจมตีอยากได้ที่สุด

ทำงานยังไง

ทุก request ต้องส่ง header `X-API-Key` ที่ตรงกับ `API_KEY` ใน `.env` ยกเว้น `/health` และ `/docs` ที่ไม่มีข้อมูลลับ สร้าง key ด้วย `secrets.token_urlsafe(32)` (ห้ามใช้ `random` กับงานด้านความปลอดภัย เพราะเดาได้)

การออกแบบ

ส่วน	ทำไม
ไม่ได้ตั้ง `API_KEY` → **503**	**fail closed**: ลืมตั้งค่าต้องปิดทุกอย่าง ไม่ใช่เปิดโล่ง
`secrets.compare_digest`	เทียบโดยใช้เวลาเท่ากันเสมอ ถ้าใช้ `==` ผู้โจมตีจับเวลาแล้วเดา key ทีละตัวอักษรได้ (**timing attack**)
`APIKeyHeader`	ทำให้หน้า `/docs` มีปุ่ม Authorize
`dependencies=[Depends(...)]`	ตรวจ key ก่อนเข้าฟังก์ชัน ไม่ผ่านก็ไม่แตะฐานข้อมูล

test

`TestClient` เรียก API โดยไม่ต้องรัน uvicorn และ `monkeypatch.setenv` ตั้งค่า environment เฉพาะใน test นั้น เขียน test ก่อน → `3 failed (assert 200 == 401)` → แก้ → ผ่าน · ต้องเพิ่ม `httpx` ใน `requirements-dev.txt` เพราะบน GitHub Actions ไม่มี

ไฟล์: `main.py`, `.env.example`, `requirements-dev.txt`, `tests/test_api_auth.py`

3. C3.3: สั่งสแกนผ่านคิวงาน

ทำไมต้องมีคิว

สแกนครบ 4 แบบใช้เวลา 5–15 นาที แต่ HTTP request ปกติ timeout ภายในไม่กี่สิบวินาที ถ้าให้ API สแกนเองระหว่างที่คนเรียกรอ คนเรียกจะค้างจน timeout และถ้าหลายคนสั่งพร้อมกัน API จะค้างทั้งตัว จึงแยกงานเป็น 2 ส่วน: **API** แค่รับคำสั่งแล้วตอบทันที ส่วน **worker** คอยดึงงานจากคิวมาสแกน

ทำไมเขียนคิวเองแทน RQ / Celery

library คิวยอดนิยมต้องใช้ `fork` ของ Linux ทำงานบน Windows ได้ไม่เต็มที่ และย้าย worker ไปรันใน Docker ก็ไม่ได้ เพราะ Semgrep, Trivy และ Docker ของ ZAP ติดตั้งอยู่บน Windows คิวที่เขียนเองด้วย Redis list ใช้ประมาณ 40 บรรทัด และเห็นการทำงานทุกบรรทัด

3 ขั้น a: แยกแกนกลาง `run_full_pipeline()`

ปัญหา

ขั้นตอน "สแกน → CWE → EPSS → Gemini → รายงาน" อยู่ใน `main()` ปนกับ argparse และ `run_scan_job()` สร้าง ScanJob แล้วสแกนทันที แต่ API ต้อง**สร้างก่อนแล้วตอบ job_id กลับทันที** ให้ worker สแกนทีหลัง

แก้อย่างไร

เดิม	ใหม่
`run_scan_job()`	`create_scan_job()` สร้างสถานะ `queued` + `execute_scan_job()` เปลี่ยนเป็น `running` แล้วสแกน
ขั้นตอนใน `main()` ของ CLI	`pipeline.py` → `run_full_pipeline()` รับพารามิเตอร์ธรรมดา ไม่ผูกกับ argparse
`triggered_by="manual"`	ส่งเข้ามาได้: `cli`, `api`, `web:<user>`

หลักการ

ทุกช่องทาง (CLI, worker) เรียกแกนกลางตัวเดียวกัน ถ้าแก้วิธีสแกน ทุกช่องทางได้การแก้พร้อมกัน · ระหว่างย้ายโค้ดยังไม่ commit เพราะโค้ดอยู่ในสภาพพังครึ่งทาง (`ImportError`) commit เมื่อทุกอย่างกลับมาใช้ได้แล้วเท่านั้น

ไฟล์: `pipeline.py` (ใหม่), `save_findings.py`, `run_pipeline.py`

4 ขั้น b: คิวใน Redis

Redis list

`RPUSH` = API ใส่งานท้ายคิว · `BLPOP` = worker หยิบงานหน้าสุด ถ้าคิวว่างจะ**รอ**จนมีงาน (B = Blocking) · งานที่ใส่ก่อนได้ทำก่อน (**FIFO**) · ใช้ Redis แทนตารางใน Postgres เพราะ `BLPOP` รอได้โดยไม่ต้องถามซ้ำทุกวินาที

`scan_queue.py`

แปลงงานเป็น JSON (Redis เก็บได้แค่ข้อความ) ใส่แค่ `scan_job_id` + path/image/url ส่วนรายละเอียดอื่น worker อ่านจากตาราง `scanjob` · `client=None` ทำให้ test ส่ง Redis ปลอมเข้ามาได้

test ด้วย FakeRedis

class ที่มี `rpush` / `blpop` ชื่อเดียวกับของจริงแต่เก็บใน dict ทำให้ GitHub Actions test ได้โดยไม่ต้องมี Redis · เทคนิคนี้เรียก **dependency injection**: ฟังก์ชันรับของที่ต้องใช้เข้ามาแทนการสร้างเอง

ไฟล์: `scan_queue.py` (ใหม่), `tests/test_scan_queue.py` (ใหม่), `requirements.txt` (เพิ่ม `redis`), `.env.example` (เพิ่ม `REDIS_URL`)

5 bug: `TimeoutError` ตอนคิวว่าง เจอจากการลองกับของจริง

อาการ

เรียก `wait_for_scan()` ตอนคิวว่าง แทนที่จะได้ `None` กลับได้ `redis.exceptions.TimeoutError: Timeout reading from socket`

สาเหตุ

มีนาฬิกา 2 ตัวตั้งไว้ 5 วินาทีเท่ากัน: `BLPOP timeout=5` (Redis รอ 5 วินาทีแล้วตอบว่าว่าง) และ `socket_timeout` ค่าเริ่มต้นของ redis-py 8.1 (Python รอคำตอบ 5 วินาที) ครบพร้อมกัน Python เลยยอมแพ้ก่อนที่คำตอบ "ว่าง" จะมาถึง ผมทดลองยืนยัน: ไม่ตั้งค่า → error หลัง 5.0 วินาที, `socket_timeout=15` → `None`

แก้

ตั้ง `socket_timeout=30` ให้ Python รอนานกว่า Redis เสมอ

บทเรียน

test ด้วย FakeRedis จะไม่มีวันเจอ bug นี้ เพราะของปลอมไม่มี `socket_timeout` **test อัตโนมัติไม่ได้แทนการลองกับของจริง**

6 ขั้น c: `worker.py`

การทำงาน

วนรองานจากคิว (สูงสุด 5 วินาทีแล้วกลับมาเช็กว่าถูกสั่งหยุดหรือยัง) → ได้งาน → อ่าน ScanJob → ต้องเป็น `queued` → `run_full_pipeline()` → รองานถัดไป

กันพัง

สถานการณ์	worker ทำอะไร
ScanJob ถูกลบไปแล้ว	ข้าม
งานเดียวกันอยู่ในคิว 2 ครั้ง	ครั้งที่ 2 ข้าม เพราะสถานะไม่ใช่ `queued`
สแกนพังกลางทาง	ตั้งเป็น `failed` พิมพ์ error แล้วรองานถัดไป
Redis ล่ม	รอ 5 วินาทีแล้วลองใหม่
กด Ctrl+C	หยุดอย่างเรียบร้อย (แต่ถ้ากดระหว่างสแกน งานจะค้าง `running`)

`--queue` ใน CLI

สร้าง ScanJob แล้วส่งเข้าคิวแทนการสแกนเอง จบทันที

ผลทดสอบ

ใส่ ScanJob 39 เข้าคิวตอน worker ปิดอยู่ พอเปิด worker ก็หยิบไปทำทันที **งานในคิวไม่หายแม้ worker จะดับ** · ข้อความ `GLib-GIO-WARNING` ที่ขึ้นตอนสร้าง PDF มาจาก GTK ที่ WeasyPrint ใช้ ไม่มีผลกับรายงาน

ไฟล์: `worker.py` (ใหม่), `run_pipeline.py`

4. C3.3 ขั้น d: กันการโจมตีผ่าน `POST /scans`

ตอนใช้ CLI คนพิมพ์คำสั่งคือตัวเราเอง แต่ API รับค่าจากใครก็ได้ที่มี API key จึงต้องคิดว่าถ้าเขาส่งค่าแปลกๆ มา จะเกิดอะไรขึ้น ขั้นนี้คือ**ครั้งแรกที่คนข้างนอกส่งค่าเข้ามาสั่งให้ระบบรันโปรแกรม**

ส่งมาแบบนี้	อันตราย	ชื่อ
`repo_path = C:\Users\Jacop`	scanner อ่านไฟล์ส่วนตัว ผลไปอยู่ในรายงาน	อ่านไฟล์นอกขอบเขต
`Targets\..\..\Users`	ดูเหมือนอยู่ใน Targets แต่ `..` พาออกไป	**path traversal**
`target_url = http://192.168.1.1`	ZAP ยิงเครื่องในเครือข่ายภายใน	**SSRF**
URL เว็บคนอื่น	ใช้ระบบเราสแกนโดยไม่ได้รับอนุญาต (ผิดกฎหมาย)	
`image = --help`	Trivy อ่านเป็น option แทนชื่อ image	**argument injection**

7 `scan_policy.py` : allowlist

allowlist ไม่ใช่ blocklist

blocklist (ห้าม `C:\Windows`, ห้าม `192.168.*`, ...) ห้ามไม่มีวันครบ allowlist อนุญาต**เฉพาะ**ที่ระบุ นอกนั้นห้ามหมด ของใหม่ที่ไม่ได้คิดไว้ก็ถูกห้ามอัตโนมัติ · ค่าเก็บใน `.env` : `SCAN_ALLOWED_REPO_ROOTS` และ `SCAN_ALLOWED_HOSTS` · ไม่ตั้งค่า = ห้ามทั้งหมด (fail closed)

จุดสำคัญในโค้ด

ส่วน	ทำไม
`os.path.realpath()`	แปลง `..` เป็น path จริง**ก่อน**เทียบ
`os.path.commonpath()`	ปลอดภัยกว่า `startswith` ที่จะปล่อย `C:\Allprojects\TargetsEvil` ผ่าน
`os.path.normcase()`	Windows ไม่สนตัวพิมพ์ใหญ่เล็ก
`urlsplit().hostname`	เทียบเฉพาะ host: `host.docker.internal.evil.com` ไม่ผ่าน
รับเฉพาะ `http` / `https`	ปฏิเสธ `ftp://`, `file:///C:/Windows/win.ini`
`resolve_scans()`	ย้ายกฎ "สแกนอะไรต้องมีค่าอะไร" ออกจาก argparse มาให้ API ใช้ได้

test 17 ตัว

ใช้ `tmp_path` (โฟลเดอร์ชั่วคราวที่ pytest สร้างให้) จึงรันบน Linux ได้ และ `pytest.raises(ValueError)` เช็กว่าค่าผิดถูกปฏิเสธจริง ครอบคลุม: นอก root, `..`, ชื่อขึ้นต้นเหมือนกัน, ไม่ตั้งค่า, host อื่น, scheme อื่น

ไฟล์: `scan_policy.py` (ใหม่), `tests/test_scan_policy.py` (ใหม่), `.env.example`

8 `POST /scans` : 5 ด่านก่อนใส่คิว

```
1. API key / session          -> 401
2. Pydantic: format           -> 422  (bad name, image "--x", unknown scan type)
3. resolve_scans: inputs      -> 400  (dast without target_url)
4. repo in allowlist + exists -> 400
5. host in allowlist          -> 400
   -> ScanJob(queued) -> RPUSH -> 202 {scan_job_id, status_url}
```

Pydantic model

`class ScanRequest(BaseModel)` กำหนดหน้าตา JSON · `pattern=r"^[A-Za-z0-9]..."` image ห้ามขึ้นต้นด้วย `-` และห้ามมีช่องว่างหรือ `;` · `Literal[...]` รับได้แค่ 4 ประเภทสแกน · `max_length` กันข้อความยาวเกิน

รายละเอียดที่ช่วยกันพัง

ส่ง path ที่ผ่าน `realpath` แล้วให้ worker ไม่ใช่ค่าดิบ · ถ้าใส่คิวไม่ได้ (Redis ล่ม) ตั้ง ScanJob เป็น `failed` ทันทีแล้วตอบ 503 ไม่ปล่อยให้ค้าง `queued` ตลอดไป · ตอบ **202 Accepted** = รับงานแล้วแต่ยังไม่เสร็จ

test 9 ตัว

ทุกตัวถูกปฏิเสธก่อนถึงฐานข้อมูล จึงรันบน GitHub ได้ · `@pytest.fixture(autouse=True)` ตั้งค่า environment ให้ทุก test ในไฟล์อัตโนมัติ

ทดสอบจริง

สั่งผ่าน `/docs` ได้ 202 ทันที worker เริ่มสแกนภายในไม่กี่วินาที สถานะเปลี่ยน `queued → running → completed` และลองโจมตีระบบตัวเองทั้ง 4 แบบ ถูกปฏิเสธทั้งหมด

ไฟล์: `main.py`, `tests/test_api_scans.py` (ใหม่)

9 C3.4 ดาวน์โหลดรายงาน

Endpoint

`GET /scan-jobs/{id}/report` และ `GET /projects/{name}/report` (ฉบับล่าสุด) ใช้ข้อมูลจากตาราง `reports` ที่มีอยู่แล้ว ตาราง `reports` ไม่มี `project_id` จึงต้อง `join` ผ่าน `scanjob`

อันตราย

ถ้าส่งไฟล์ตาม `file_path` ในฐานข้อมูลตรงๆ แล้วค่านั้นถูกเปลี่ยนเป็น `.env` API จะส่งรหัสผ่านทั้งหมดออกไป

กัน

ใช้ `is_repo_allowed(path, [REPORTS_DIR])` ส่งได้เฉพาะไฟล์ใต้ `reports/` หลักนี้เรียก **defense in depth**: แม้เชื่อว่าข้อมูลในฐานข้อมูลถูกต้อง ก็ยังตรวจซ้ำก่อนใช้ · ชื่อไฟล์ตอนดาวน์โหลดเป็น `NodeGoat_scanjob40_security_report.pdf`

ข้อสังเกต

เปิด URL ตรงๆ ใน browser ได้ 401 เพราะ browser ไม่รู้ว่าต้องส่ง header `X-API-Key` นี่คือเหตุผลที่หน้าเว็บต้องมีระบบ login

ไฟล์: `main.py`, `tests/test_api_auth.py`

5. ปรับแผน: เป้าหมายปลายทางและทางเลือกด้าน frontend

เป้าหมายที่ชัดขึ้น

ระบบต้องใช้งานได้ **3 ทาง** คือ CLI, CI/CD และ web app ที่ login ด้วย user/password เพื่อดู dashboard สั่งสแกน และพิมพ์รายงาน ทั้ง 3 ทางต้องเรียกแกนกลางตัวเดียวกัน และต้อง**ใช้เป็นผลงานสมัครงานได้**

dashboard 2 ความหมาย

	ความหมาย
dashboard ของ RQ / Celery	หน้าสำหรับนักพัฒนาดูว่าคิวมีงานค้างกี่งาน (ไม่จำเป็น)
dashboard ของระบบเรา	หน้าเว็บสำหรับผู้ใช้ดูช่องโหว่ สร้างเองบนคิวที่เขียนเองได้ เพราะสถานะทุกงานอยู่ในตาราง `scanjob`

ใช้ library หรือเขียนเอง

ส่วน	ใครทำ
hash รหัสผ่าน, ลายเซ็น cookie, สุ่ม token	📦 library เสมอ ("Don't roll your own crypto")
รับ request, template, กราฟ	📦 library (FastAPI, Jinja2/React, Chart.js)
ขั้นตอน login, หน้าตา dashboard, ฟอร์มสั่งสแกน	✍️ เขียนเอง เพราะเป็นเรื่องเฉพาะของระบบเรา

ทางเลือกด้าน frontend

ทางเลือก	ข้อดี	ข้อเสีย
⭐ **React + TypeScript**	ทักษะ frontend ที่ตลาดงานต้องการมากที่สุด แยก frontend/backend ต่อยอดได้ (แอปมือถือใช้ API เดิม)	ต้องเรียนภาษาและเครื่องมือใหม่
`fastapi-users`	เร็ว	ซ่อนทุกอย่าง ไม่ได้เรียนรู้
Streamlit / Grafana	dashboard เร็วมาก	ใส่ login และฟอร์มสั่งสแกนแบบที่ต้องการไม่ได้

ตัดสินใจ: React + TypeScript เพื่อผลงานสมัครงาน · สิ่งที่ทำให้โปรเจกต์โดดเด่นจริงคือ**ความลึกด้าน security** (fingerprint, EPSS, allowlist, Argon2, CI) จึงเพิ่มช่วง C8 สำหรับเล่าเรื่องเหล่านี้ · ระวังบทความที่สอนเก็บ token ใน `localStorage` (XSS ขโมยได้) เราใช้ cookie `HttpOnly` แทน

แผนใหม่

ช่วง	งาน
C5	React + TypeScript (Vite): login, dashboard (การ์ด กราฟ ตาราง กรอง ค้นหา), ประวัติการสแกน, ฟอร์มสั่งสแกน, ดาวน์โหลด/พิมพ์รายงาน
C6	CI/CD: `--fail-on critical`, ตัวอย่าง workflow ให้โปรเจกต์อื่นเรียกใช้, CI ฝั่ง frontend
C7	Production: Docker image ของ API / worker / frontend, `docker compose up` คำสั่งเดียว, HTTPS, logging, backup อัตโนมัติ, เก็บหนี้เทคนิค
C8	Portfolio: README พร้อมภาพหน้าจอและแผนภาพ, threat model, วิดีโอสาธิต, ข้อมูลตัวอย่าง

6. C4: ระบบ login

10 C4.1 ตาราง `users` + Argon2

ห้ามเก็บรหัสผ่านจริง

วิธีเก็บ	ปัญหา
❌ ข้อความตรงๆ	ฐานข้อมูลหลุด = ได้รหัสของทุกคน และคนส่วนใหญ่ใช้รหัสเดียวกันหลายเว็บ
❌ encrypt	ได้ key ไปด้วยก็ถอดกลับได้
❌ MD5 / SHA-256	เร็วเกินไป เดาได้หลายพันล้านครั้งต่อวินาที
☑ Argon2id	ตั้งใจให้ช้าและใช้หน่วยความจำ (64 MB ต่อครั้ง) OWASP แนะนำอันดับแรก

hash เป็นทางเดียว ย้อนกลับไม่ได้ ตอน login ให้ hash รหัสที่พิมพ์แล้วเทียบ · **salt** สุ่มทุกครั้ง รหัสเดียวกันได้ hash ต่างกัน ใช้ตารางที่คำนวณไว้ล่วงหน้าเทียบไม่ได้

`auth.py`

`hash_password()`, `verify_password()` (แปลง error ทุกแบบเป็น `False` hash เสียหายก็ไม่ crash)

ตาราง `users`

`username` (unique), `password_hash` (ชื่อบอกชัดว่าเป็น hash), `is_active` (ปิดบัญชีแทนการลบ), `created_at`, `last_login_at`

Alembic

backup ด้วย `pg_dump` ก่อน → `alembic revision --autogenerate` → **อ่านไฟล์ migration ก่อนรัน** (autogenerate เดาผิดได้ เช่น เปลี่ยนชื่อคอลัมน์จะกลายเป็นลบแล้วสร้างใหม่ ข้อมูลหาย) → `alembic upgrade head` → `alembic check`

ไฟล์: `auth.py` (ใหม่), `models.py`, `alembic/versions/1df495aea930_add_users_table.py` (ใหม่), `tests/test_auth.py` (ใหม่), `requirements.txt` (เพิ่ม `argon2-cffi==25.1.0`)

11 C4.2 `manage_users.py`

ทำไมไม่มีหน้าสมัครสมาชิก

เป็นเครื่องมือภายในที่ดูข้อมูลช่องโหว่ได้ ผู้ดูแลระบบต้องเป็นคนสร้างบัญชีให้เท่านั้น

คำสั่ง

`create`, `passwd`, `disable`, `enable`, `list` (ใช้ `add_subparsers` แบบ `git add`)

กฎรหัสผ่าน (OWASP / NIST)

ยาว 12–128 ตัว **ไม่บังคับ**ตัวใหญ่/ตัวเลข/สัญลักษณ์ (ทำให้คนตั้ง `Password1!`) passphrase ภาษาไทยยาวๆ แข็งแรงกว่า · จำกัด 128 ตัวกันการส่งรหัสยาวมากมาถ่วงเครื่อง

`getpass`

พิมพ์แล้วไม่ขึ้นจอ และไม่อยู่ในประวัติคำสั่ง (ถ้ารับผ่าน `--password` กดลูกศรขึ้นก็เห็น) ถามซ้ำ 2 ครั้งกันพิมพ์ผิด

ผล

ในฐานข้อมูลเก็บเป็น `$argon2id$v=19$m=65536,t=3,p=4$...`

ไฟล์: `manage_users.py` (ใหม่), `auth.py` (`password_problem`)

12 C4.3 login ด้วย session cookie

ทำงานยังไง

`POST /auth/login` → ตรวจด้วย Argon2 → `Set-Cookie: vc_session=<base64 {"user_id": 1}>.<ลายเซ็น>` → browser แนบ cookie ให้อัตโนมัติทุก request · มี `/auth/me` และ `/auth/logout`

การป้องกัน

การโจมตี	วิธีกัน
XSS ขโมย cookie	`HttpOnly` : `document.cookie` ได้ `''`
CSRF	`SameSite=Strict` : browser ไม่แนบ cookie กับ request ที่มาจากเว็บอื่น
ปลอม cookie	ลายเซ็นด้วย `SESSION_SECRET` (`itsdangerous`) แก้ cookie แล้วได้ 401
brute force	`LoginRateLimiter` : ผิด 5 ครั้งใน 15 นาที → 429
user enumeration	ข้อความเดียวกันทุกกรณี และตรวจกับ `DUMMY_PASSWORD_HASH` ตอนไม่มี user ให้เวลาตอบเท่ากัน
บัญชีถูกปิด	เช็ก `is_active` จากฐานข้อมูลทุก request
ลืมตั้ง secret	สุ่มใหม่ตอนเปิด API (ทุกคนต้อง login ใหม่หลัง restart) ไม่ใช้ค่าตายตัวอย่าง `"changeme"`

ข้อควรรู้

cookie ของ Starlette **อ่านได้** (base64 ไม่ใช่การเข้ารหัส) แต่**แก้ไม่ได้** จึงห้ามเก็บความลับในนั้น · "session" มี 2 ความหมายในโค้ด: `Session` ของฐานข้อมูล (ตั้งชื่อ `db` ใน endpoint ใหม่) กับ `request.session` ของ cookie · `SESSION_HTTPS_ONLY=true` ตอนขึ้น production

test

`LoginRateLimiter` ใช้นาฬิกาปลอม (`FakeClock`) ไม่ต้องรอ 15 นาทีจริง · ส่ง cookie ปลอม `eyJ1c2VyX2lkIjogMX0=.fake.signature` ต้องได้ 401

ไฟล์: `main.py` , `auth.py` , `requirements.txt` (เพิ่ม `itsdangerous==2.2.0`), `.env.example` , `tests/test_api_login.py` (ใหม่)

13 C4.4 รับได้ทั้ง API key และ session

`require_auth()`

มี header `X-API-Key` → ต้องผ่าน key เท่านั้น (key ผิดไม่ "ลองดู cookie แทน") คืน `"api"` · ไม่มี → ตรวจ session คืน `"web:<username>"` · ไม่มีทั้งคู่ → 401

ผล

CI/CD และหน้าเว็บใช้ endpoint ชุดเดียวกัน · `triggered_by` บันทึกว่าใครสั่งสแกน (**audit trail**) · ต้องรับเป็นพารามิเตอร์ของ endpoint แทน `dependencies=[...]` เพราะ `dependencies` ตรวจแล้วทิ้งค่าที่คืนกลับมา

เครื่องมือใหม่

`Ctrl+H` ใน VS Code ค้นหาและแทนที่ทั้งไฟล์ (เปลี่ยน 6 จุดพร้อมกัน)

ไฟล์: `main.py`

7. ปัญหาที่เจอระหว่างทาง และบทเรียน

> **เหตุการณ์สำคัญ: ไฟล์ที่ไม่ได้ commit ทำให้ CI พัง**
> ตอนขั้น b terminal อยู่ในโฟลเดอร์ `tests` ทำให้ `git add scan_queue.py` หาไฟล์ไม่เจอ `scan_queue.py` กับ test จึงไม่ถูก commit แต่ `main.py` , `worker.py` , `run_pipeline.py` ที่ import มันถูก push ขึ้นไปแล้ว
> ในเครื่องใช้ได้เพราะมีไฟล์อยู่ แต่ใครที่ clone repo ไปจะรันไม่ได้ และ GitHub Actions พังด้วย `ModuleNotFoundError` · เจอตอนอ่าน `git status` แล้วเห็น Untracked files ค้างอยู่ · บทเรียน: **อ่าน git status ให้ครบทุกส่วน, Actions ขึ้น ❌ ต้องแก้ทันที, "ในเครื่องใช้ได้" ไม่ได้แปลว่าบน GitHub ใช้ได้**

อาการ	สาเหตุ	วิธีแก้ / บทเรียน
เปิด `http://127.0.0.1:8000` ได้ 404	ไม่มีหน้า `/`	ต้องเปิด `/docs` · log ของ uvicorn บอก URL และรหัสผลทุก request (200, 404, 422, 500)
แก้ไฟล์แล้วไม่มีอะไรเปลี่ยน	ยังไม่ได้กด Ctrl+S หรือวางโค้ดไม่ครบ	ดู ● บนแท็บ, `dir` ดูขนาด/เวลา, `git status`
`collected 0 items`	copy มาแค่บรรทัดเดียว (ข้อความผมกำกวม)	copy ทั้งกล่องด้วยปุ่มมุมกล่อง
`file or directory not found: tests\tests\...`	terminal อยู่ในโฟลเดอร์ `tests`	`cd ..` · ดู prompt ก่อนทุกครั้ง
`--repo: expected one argument`	คำสั่งยาวถูกตัดเป็น 2 บรรทัดตอน copy	เช็กว่าอยู่บรรทัดเดียว หรือแบ่งด้วย backtick `` ` `` ท้ายบรรทัด (prompt จะขึ้น `>>`)
worker ไม่ได้รับงาน	รันทุกอย่างใน terminal เดียว	เปิด 2 terminal: worker ค้างไว้อันหนึ่ง คำสั่งอื่นอีกอันหนึ่ง
`NameError: name 'time' is not defined` ใน test	วาง class จริงลงในไฟล์ test	โค้ดจริงมี `class` / `def` ทำงาน, ไฟล์ test มี `def test_...` กับ `assert`
`get_current_user` ซ้ำ 2 ชุด	กด Ctrl+C แทน Ctrl+X ตอนย้าย	Python ให้ตัวหลังทับตัวแรกโดยไม่เตือน อันตรายตอนแก้ทีหลัง
เส้นหยัก `could not be resolved`	VS Code ใช้ Python คนละตัวกับ venv	Ctrl+Shift+P → Python: Select Interpreter → venv
`.github/workflows/tests.yml` ถูกแก้	เผลอเพิ่มช่องว่างท้ายบรรทัด	`git restore` · ดู `git diff` ก่อน commit
ผมนับ test ผิด (58 แทน 56)	นับ parametrize ผิด	ตัวเลขจาก pytest คือความจริง

คำสั่งและแนวคิดใหม่

คำสั่ง / แนวคิด	ใช้เมื่อ
`uvicorn main:app --reload`	รัน API (แก้โค้ดแล้ว restart เอง แต่แก้ `.env` ต้อง restart เอง)
`python worker.py`	รัน worker รองานจากคิว
`python run_pipeline.py ... --queue`	ส่งงานเข้าคิวจาก CLI
`docker exec secpipeline-redis redis-cli LRANGE scan_queue 0 -1`	แอบดูงานในคิว (`LLEN` = จำนวน)
`python manage_users.py create|passwd|disable|enable|list`	จัดการ user
`alembic revision --autogenerate -m "..."`	สร้าง migration จาก `models.py` (อ่านก่อนรัน)
F12 → Application → Cookies / Console	ดู cookie, ทดสอบ `document.cookie`

รหัส HTTP	200 สำเร็จ · 202 รับงานแล้ว · 400 ค่าใช้ไม่ได้ · 401 ไม่ได้ยืนยันตัวตน · 404 ไม่พบ · 422 รูปแบบผิด · 429 ถี่เกิน · 503 ระบบยังไม่พร้อม
fail closed · allowlist · defense in depth · dependency injection	หลักการออกแบบที่ใช้ซ้ำทั้งรอบ

8. สถานะงาน และสิ่งที่ยังเหลือ

ความคืบหน้า

ช่วง	สถานะ
ขั้นที่ 1–4 (pipeline, git, test, CI, EPSS, Gemini)	เสร็จ
C3 REST API + คิว + worker + รายงาน	เสร็จ
C4 ระบบ login	เสร็จ
C5 React frontend	ถัดไป
C6 CI/CD · C7 Production · C8 Portfolio	ยังไม่ทำ

หนี้เทคนิคที่จดไว้ (แก้ใน C7)

เรื่อง	รายละเอียด
Redis ไม่มีรหัสผ่าน	`docker-compose.yml` เปิด port 6379 ให้ทุก interface ต้องจำกัดเป็น `127.0.0.1` และตั้งรหัส
กด Ctrl+C ระหว่างสแกน	ScanJob ค้าง `running` (เหมือน ScanJob 18–26 ที่ยังค้างอยู่)
ตัวจำกัด login อยู่ในหน่วยความจำ	restart แล้วหาย และใช้กับ API หลาย instance ไม่ได้ ควรย้ายไป Redis
cookie เพิกถอนไม่ได้	session เก็บใน cookie ฝั่ง client ถ้าถูกขโมย ใช้ได้จนหมดอายุ (8 ชม.) ยกเว้นปิดบัญชี ควรเก็บ session ฝั่ง server ใน Redis
`request.client.host`	เมื่ออยู่หลัง reverse proxy จะเห็น IP ของ proxy ต้องตั้งค่า forwarded headers
worker / API ใช้ path `reports` แบบ relative	ต้องรันจากโฟลเดอร์โปรเจกต์ ควรใช้ path เต็ม
`datetime.utcnow()`	deprecated ควรใช้ `datetime.now(timezone.utc)`
warning `httpx2`	Starlette จะเลิกรองรับ `httpx` ใน TestClient
`is_repo_allowed` ใช้กับ reports ด้วย	ชื่อไม่ตรงความหมาย ควรเปลี่ยนเป็น `is_path_inside`

Commit ในรอบนี้

commit	ตอน	ข้อความ
`d629796`	C3.1	Add read-only API for projects, findings and scan jobs
`49da1cd`	C3.2	Require API key for API endpoints
`ee2389f`	C3.3a	Extract run_full_pipeline so CLI and worker share one code path
`c3b8e25`	C3.3c	Add worker that runs queued scans and --queue option for the CLI
`96087f6`	C3.3d	Add scan policy: allowlist repo folders and target hosts
`61c74c8`	C3.3d	Add POST /scans that validates input and queues scans for the worker
`ca658dc`	C3.4	Add report download endpoints restricted to the reports folder
`b9da171`	C4.1–2	Add users table with Argon2 password hashing and manage_users CLI
`e1ebc70`	C3.3b	Add missing scan_queue module and its tests (ตกหล่น ดูหัวข้อ 7)
`a6ca01a`	C4.3	Add session login with HttpOnly cookie, rate limiting and timing-safe checks

`ab2ef86`	C4.4	Accept either API key or web session on API endpoints

ไฟล์ที่เปลี่ยนในรอบนี้

ไฟล์	สถานะ
`main.py`	**เขียนใหม่** API 11 endpoint, API key, session, login
`pipeline.py`, `worker.py`, `scan_queue.py`, `scan_policy.py`, `auth.py`, `manage_users.py`	**ใหม่**
`save_findings.py`, `run_pipeline.py`, `models.py`	**แก้**
`alembic/versions/1df495aea930_add_users_table.py`	**ใหม่**
`requirements.txt`	**เพิ่ม** uvicorn, redis, argon2-cffi, itsdangerous
`requirements-dev.txt`	**เพิ่ม** httpx
`.env.example`	**เพิ่ม** API_KEY, REDIS_URL, SCAN_ALLOWED_*, SESSION_*
`tests/`	**ใหม่** test_api_auth, test_api_scans, test_api_login, test_scan_policy, test_scan_queue, test_auth · รวม 83 test
ฐานข้อมูล	**ใหม่** ตาราง `users` + user `admin`